



范式与规范化

“Never underestimate the power of a well-written query.”

井明

数据科学与计算机学院

jingming@sdu.edu.cn

2025年春



山东女子学院
Shandong Women's University

课程导入案例：在线商城数据库的“混乱”设计

场景背景

你是某互联网公司的数据库设计师，团队正构建一个电商系统，需要设计订单数据的存储结构。初学数据库的实习生交给你了以下设计方案：

订单ID	客户名	客户电话	产品1名称	产品1价格	产品2名称	产品2价格
001	张三	13812345678	鼠标	50	键盘	80
002	李四	13988887777	显示器	800	-	-

思考：

1. 你觉得这个设计怎么样？能用吗？
2. 如果你是数据库管理员，看到这样的数据结构，你会有什么感受？

问题分析

订单ID	客户名	客户电话	产品1名称	产品1价格	产品2名称	产品2价格
001	张三	13812345678	鼠标	50	键盘	80
002	李四	13988887777	显示器	800	-	-

虽然能运行，但你马上发现了几个设计漏洞：

1. 字段重复且有限制：产品数量被硬编码为2件，多了放不下，少了字段浪费。
2. 数据冗余：如果客户张三下了10个订单，他的姓名和电话将重复出现10次。
3. 不一致风险：若一次写成“张三”，另一次写成“张3”，系统会当成两个人。
4. 更新困难：客户更换手机号时，要修改10条记录，一不小心就改漏了。
5. 插入困难：若暂未下单的客户想注册账号，无法插入记录（因为没有订单ID）。
6. 删除异常：删除最后一个订单可能导致客户信息也一并丢失。

问题提出

我们该如何重新设计这个数据库，让数据结构更合理、更新更方便、冗余更少？

课程主题

这就是“数据库规范化（Normalization）”的目的，也是我们今天要学习的核心内容：

通过范式（Normal Forms）对关系模式进行约束，逐步消除冗余、避免异常，让数据库结构更“干净”、更易维护。

教学目标

通过这个案例将理解：

- 不规范的数据结构会带来什么后果；
- 为什么“看起来能存就行”的设计远远不够；
- 范式是一种结构优化思想，不仅理论上重要，实际开发中也非常关键。

什么是范式（ Normal Form ）？

✅ 定义 (Definition) :

范式 (Normal Form) 是关系数据库中用于衡量表结构规范程度的标准形式。

通过一系列规则对关系模式进行约束，目的是减少数据冗余，防止数据异常 (插入、删除、更新)。

为什么需要范式？

在数据库设计中，如果关系结构设计不合理，可能导致以下问题：

问题类型	描述
冗余数据	数据重复，浪费存储空间
插入异常	因部分信息缺失而无法插入新记录
删除异常	删除记录时意外丢失有用信息
更新异常	更新操作不一致，导致数据错误

这些问题可以通过**规范化（Normalization）**处理——即将表转换为一种或多种“范式”。



范式的层级

🌟 常见的范式层级（依次递进）：

范式	全称	核心要求
1NF	第一范式	属性值必须是原子值（不可再分）
2NF	第二范式	满足1NF，且每个非主属性完全依赖主键
3NF	第三范式	满足2NF，且非主属性不依赖于其他非主属性
BCNF	Boyce-Codd范式	每个决定因素都是候选键
4NF	第四范式	消除多值依赖
5NF	第五范式（PJ/NF）	消除连接依赖（Join Dependency）

类比记忆:

把范式想象成“整理房间”:

- 1NF: 每件物品单独放好 (不能把衣服和鞋子塞一个抽屉);
- 2NF: 分类整理 (衣服放衣柜, 鞋子放鞋柜);
- 3NF: 避免冗余 (标签别贴重复, 避免“这件外套是蓝色”写三次);
- 更高范式: 结构更加系统化, 适用于更复杂的数据库应用场景。

✚ 第一范式（1NF）：属性值必须原子化

✓ 定义（Definition）：

关系中的每一个属性都必须是不可再分的原子值（Atomic Value）。

简单来说，一张表的每个格子里只能存一个值，不能有列表、集合、多个字段组合等复杂结构。

为什么需要1NF?

如果一个字段中存放多个值，会导致：

问题类型	示例	后果
数据冗余	一个字段存放多个电话号码	查询困难、模糊匹配不准确
更新困难	改一个值需要解析字段并重组	SQL 操作复杂，容易出错
插入异常	没有课程不能插入空值	表结构阻碍了数据的录入
删除异常	删除一门课可能导致整个记录被删	意外丢失其它相关信息

1NF 的目标:

确保每个属性只存储一个值，结构清晰、便于操作。



示例与转换

✗ 未满足1NF的表（非原子值）：

学号	姓名	联系电话
1001	张三	13800001111,13999992222
1002	李四	13788883333

问题：电话字段中存了多个值。

✓ 转换为1NF的表：

学号	姓名	联系电话
1001	张三	13800001111
1001	张三	13999992222
1002	李四	13788883333

做法：将每个电话分开存储，保持每个字段“原子”。

✓ 示例二：课程字段不原子

✗ 非1NF:

学号	姓名	所选课程
2001	王五	数据结构, 操作系统, 数据库

✓ 满足1NF:

学号	姓名	所选课程
2001	王五	数据结构
2001	王五	操作系统
2001	王五	数据库



图示说明:

📎 表格中属性值是否原子?

情况	满足1NF?	原因
课程 = 数据结构	✓	单一值
课程 = 数据结构,数据库	✗	非原子值 (多个数据组合)

练习题 (1NF)

题目： 下列表格中，哪些不满足第一范式？为什么？应该如何转换？

员工ID	姓名	技能
E001	王雷	Java, Python
E002	赵敏	SQL
E003	李云	HTML, CSS, JS

✅ 答案与解析:

答案: E001 和 E003 的“技能”字段不满足第一范式, 因为它包含了多个值。

转换后:

员工ID	姓名	技能
E001	王雷	Java
E001	王雷	Python
E002	赵敏	SQL
E003	李云	HTML
E003	李云	CSS
E003	李云	JS

✖ 第二范式（2NF）：消除部分依赖

✓ 定义（Definition）：

一个关系模式满足第一范式（1NF），并且其中所有非主属性都完全函数依赖于候选键（主键），则该关系满足第二范式（2NF）。

简单来说，2NF 要求每个非主属性都要完全依赖于主键，而不是主键的一部分。

如果一个表的主键是组合键（由多个字段组成），那么所有非主属性都必须依赖于整个组合键，而不是其中的某个部分。

知识点回顾：

1. 候选键（Candidate Key）：唯一标识元组的字段（或字段组合），可以是单个字段或多个字段的组合。
2. 主键（Primary Key）：从候选键中选出的一个，用于唯一标识表中的每一行。
3. 非主属性（Non-Primary Attribute）：不属于主键的属性。

核心术语：

- 完全依赖：非主属性依赖整个主键，而不是主键的一部分。
 - 部分依赖：非主属性只依赖主键的一部分（当主键是组合键时）。
 - 主键：唯一标识元组的字段（或字段组合）。
- ✅ 2NF 解决的是：组合主键导致的部分依赖问题。

关键术语：

- 函数依赖（Functional Dependency）：如果属性A的值决定了属性B的值，则称B函数依赖于A，记作 $A \rightarrow B$ 。
- 完全函数依赖（Full Functional Dependency）：如果属性B函数依赖于属性A，并且B不依赖于A的任何真子集，则称B完全函数依赖于A，记作 $A \xrightarrow{f} B$ 。
- 部分函数依赖（Partial Functional Dependency）：如果属性B函数依赖于属性A，但B也依赖于A的某个真子集，则称B部分函数依赖于A，记作 $A \xrightarrow{p} B$ 。
- 平凡依赖：若 $X \rightarrow Y$ ，且Y是X的子集（对任一关系模式，平凡函数依赖必然成立），就是平凡函数依赖。
- 非平凡依赖：若 $X \rightarrow Y$ ，且Y不是X的子集，就是非平凡函数依赖。

为什么需要2NF?

如果一个表有组合主键，但某些非主属性只依赖主键的一部分，就会产生：

问题类型	后果
数据冗余	非主属性重复出现在多个记录中
更新异常	修改一处数据，需要同步修改多处
插入异常	如果某部分信息缺失，就无法录入相关属性
删除异常	删除一条记录时，可能丢失重要信息

示例与转换

✗ 未满足2NF的关系:

学号	课程号	课程名	分数
1001	C01	数据结构	85
1001	C02	操作系统	78
1002	C01	数据结构	88

- 主键 = (学号, 课程号)
- 非主属性:
 - 分数: 依赖于 (学号, 课程号) ✓ (完全依赖)
 - 课程名: 只依赖于 课程号 ✗ (部分依赖)

✅ 转换后满足2NF:

拆成两个关系模式:

📘 学生成绩表:

学号	课程号	分数
1001	C01	85
1001	C02	78
1002	C01	88

📘 课程信息表:

课程号	课程名
C01	数据结构
C02	操作系统

 图示说明:

- 原关系中“课程名”只依赖课程号（主键的一部分）→ 不满足2NF。
- 拆分后，每个非主属性都完全依赖主键 → 满足2NF。

✅ 示例二：图书借阅记录

✗ 未满足2NF:

读者ID	图书ID	图书名	借阅日期
R001	B001	操作系统	2024-10-01
R001	B002	数据结构	2024-10-02
R002	B001	操作系统	2024-10-03

- 主键 = (读者ID, 图书ID)
- “图书名”依赖于 图书ID → 违反2NF

✅ 分解为两个表:

- 借阅信息表 (读者ID, 图书ID, 借阅日期)
- 图书表 (图书ID, 图书名)

练习题 (2NF)

题目：下表中是否满足2NF？若不满足，请说明原因并规范化。

课程号	学号	课程名	成绩
C001	S001	数据结构	88
C002	S001	操作系统	92
C001	S002	数据结构	85

✅ 答案与解析:

- 主键为 (课程号, 学号)
- “成绩”依赖于 (课程号, 学号) ✅
- “课程名”只依赖于课程号 ❌ → 部分依赖

不满足2NF, 应该拆分为:

1. 选课成绩表 (课程号, 学号, 成绩)
2. 课程信息表 (课程号, 课程名)

✚ 第三范式 (3NF) : 消除传递依赖

✓ 定义 (Definition)

一个关系模式若满足第二范式 (2NF), 并且每个非主属性都不传递依赖于候选键, 则该关系满足第三范式 (3NF)。

简单来说, 3NF 要求非主属性不能依赖于其他非主属性。

关键术语：

- 传递依赖： $A \rightarrow B$, $B \rightarrow C$, 则 $A \rightarrow C$ 是传递依赖。
- 候选键：可以唯一标识一个元组的最小属性集。
- 非主属性：不是主键、也不是候选键的属性。

3NF 解决的问题:

如果非主属性依赖于其他非主属性，就可能出现以下问题：

问题类型	描述
冗余数据	重复出现间接信息，如部门名称
更新异常	修改一个字段需要修改多条记录
插入异常	缺少某字段导致无法插入记录
删除异常	删除一条记录可能导致丢失其他信息



示例与转换

✗ 不满足3NF的表:

学号	姓名	专业号	专业名称
1001	张三	CS01	计算机科学
1002	李四	CS02	软件工程
1003	王五	CS01	计算机科学

- 主键：学号
- “专业名称”依赖于“专业号”，而“专业号”又依赖于“学号”
- 即：学号 $\rightarrow$ 专业号 $\rightarrow$ 专业名称（传递依赖 ✗）

✔ 满足3NF后的分解:

表1: 学生表

学号	姓名	专业号
1001	张三	CS01
1002	李四	CS02
1003	王五	CS01

表2: 专业表

专业号	专业名称
CS01	计算机科学
CS02	软件工程

 类比说明：

学号是身份证 → 专业号是学籍号 → 专业名称是专业称呼

如果你已经知道身份证，就能知道学籍号，再知道专业。那“专业名称”依赖了“身份证”吗？不是直接，是间接（传递）。

🎓 示例二：员工关系表

✗ 不满足3NF:

员工ID	姓名	部门ID	部门名称
E001	张三	D01	财务部
E002	李四	D02	市场部
E003	王五	D01	财务部

- 主键：员工ID
- “部门名称”不是直接依赖员工ID，而是传递依赖于部门ID（员工ID → 部门ID → 部门名称） ✗

✔ 满足3NF后的分解:

员工表:

员工ID	姓名	部门ID
E001	张三	D01
E002	李四	D02
E003	王五	D01

部门表:

部门ID	部门名称
D01	财务部
D02	市场部

练习题 (3NF)

题目：下表是否满足第三范式？如果不满足，请说明原因并规范化。

学号	姓名	辅导员ID	辅导员姓名
S001	王亮	T01	李老师
S002	刘颖	T02	张老师
S003	赵明	T01	李老师

✅ 答案与解析:

- 主键是学号。
- “辅导员姓名”依赖于“辅导员ID”，而“辅导员ID”又依赖“学号” → 传递依赖
- 所以 不满足3NF。

应拆分为两张表:

1. 学生表 (学号, 姓名, 辅导员ID)
2. 辅导员表 (辅导员ID, 辅导员姓名)

✓ 小结

范式	解决的问题	依赖类型
1NF	属性值需为原子	无
2NF	消除对主键的部分依赖	非主属性依赖主键
3NF	消除对非主属性的传递依赖	非主属性依赖非主属性

🎓 第三范式 (3NF) 进阶讲解

一、形式化定义（理论课程讲解）

若关系模式R已属于第二范式（2NF），且其中不存在非主属性对候选键的传递依赖，则R属于第三范式（3NF）。

也可以表述为：

一个关系模式R满足3NF，若对于任意函数依赖 $X \rightarrow A$ ，满足以下至少一个条件：

1. A 是主属性（属于某个候选键）
2. X 是超级键
3. 该依赖是平凡依赖（即A属于X）

二、为什么需要第三范式？

虽然 2NF 消除了 部分依赖，但仍可能存在间接依赖/传递依赖，这可能带来：

- 更新异常（Update Anomaly）：修改一位老师名字，要改所有学生记录
- 插入异常（Insert Anomaly）：无法单独插入某个辅导员信息
- 删除异常（Delete Anomaly）：删除学生记录可能导致辅导员信息丢失

✅ 3NF 进一步增强数据的逻辑独立性，避免这些问题。



三、图示说明（结构依赖图）

原始依赖图（不满足3NF）：

学号 $\rightarrow$ 辅导员ID

辅导员ID $\rightarrow$ 辅导员姓名

$\Rightarrow$ 学号 $\rightarrow$ 辅导员ID $\rightarrow$ 辅导员姓名（传递依赖）

满足3NF后的依赖结构：

学号 $\rightarrow$ 辅导员ID

辅导员ID $\rightarrow$ 辅导员姓名

$\Rightarrow$ 分拆为两个表，传递依赖被打断

四、更多例子

✓ 例子1：项目管理表

员工ID	员工姓名	项目ID	项目经理
E001	王强	P001	李总
E002	李四	P001	李总
E003	张三	P002	赵总

- 主键为：员工ID
- 项目经理依赖于项目ID，而项目ID依赖员工ID → 传递依赖 ✗

分解为：

1. 员工表（员工ID，员工姓名，项目ID）
2. 项目信息表（项目ID，项目经理）

✓ 例子2：班级信息表

学号	姓名	班级号	班主任
S001	小明	C01	张老师
S002	小红	C02	王老师
S003	小刚	C01	张老师

- 主键为学号
- 班主任依赖于班级号，而班级号依赖学号 → 传递依赖 ✗

分解为：

1. 学生表（学号，姓名，班级号）
2. 班级表（班级号，班主任）

五、3NF 与 2NF 的对比总结

对比项	第二范式（2NF）	第三范式（3NF）
消除依赖类型	消除部分依赖	消除传递依赖
要求	所有非主属性完全依赖于主键	所有非主属性仅直接依赖主键
应用场景	主键是组合键时	存在非主属性依赖非主属性时
是否需要拆表	有组合主键时需要	有传递依赖时需要



五、练习题（进阶版）

题目：

某大学的课程选修记录如下表所示：

选课ID	学号	课程号	课程名	任课老师
R001	S001	C01	数据结构	张老师
R002	S002	C01	数据结构	张老师
R003	S001	C02	操作系统	李老师

问：该表是否满足3NF？若不满足，请说明原因并规范化。

解析:

- 主键: 选课ID
- 课程名、任课老师依赖于课程号, 而课程号依赖于选课ID (通过学号)
- 存在传递依赖 ✖

规范化:

1. 选课表 (选课ID, 学号, 课程号)
2. 课程表 (课程号, 课程名, 任课老师)

Boyce-Codd范式 (BCNF)

✅ 一、定义 (Definition)

一个关系R属于BCNF (Boyce-Codd Normal Form) , 当且仅当对于每一个非平凡的函数依赖 $X \rightarrow Y$, 都满足:

👉 X 是候选键。

换句话说:

BCNF 是第三范式 (3NF) 的加强版, 它消除了 候选键之间的函数依赖冲突问题。

! 二、BCNF vs 3NF: 区别

项目	第三范式 (3NF)	Boyce-Codd 范式 (BCNF)
非主属性 $\rightarrow$ 主属性	✓ 允许	✗ 不允许 (如果不是候选键)
要求的依赖规则	若 A 不是主属性, 只要 $A \rightarrow B$ 满足候选键	所有函数依赖左边必须是候选键
是否更严格	否	是 ($BCNF \subset 3NF$)

“ 🔍 总结一句话: 3NF 允许某些非主属性参与函数依赖, 而 BCNF 不允许。

三、为什么需要BCNF?

即使满足 3NF，仍可能因多个候选键或非主属性决定主属性的情况，导致：

- 数据冗余
- 更新异常
- 插入/删除异常



四、经典示例讲解（违背BCNF）

✗ 原始关系表：

学生	课程	教师
张三	数据库	李老师
张三	操作系统	王老师
李四	数据库	李老师

函数依赖：

- (学生, 课程) → 教师 ✓ (主键)
- 教师 → 课程 ✗ (违反BCNF)

因为：

- 教师不是候选键，但决定了课程 → 违反BCNF

✓ 分解为两个表（满足BCNF）

表1：教师-课程表

教师	课程
李老师	数据库
王老师	操作系统

表2：学生-教师表

学生	教师
张三	李老师
张三	王老师
李四	李老师

此时：

- 教师 → 课程 在 教师-课程表 中是主键 ✓
- 学生, 教师 是 学生-教师表 的主键 ✓

五、第二个例子：课程安排表

✗ 原始表：

教室	时间	课程
101	周一9:00	数据结构
101	周三9:00	数据库
102	周一9:00	操作系统

函数依赖：

- 教室 + 时间 → 课程 ✓ (主键)
- 课程 → 教室 ✗ (违反BCNF)

课程 → 教室不是候选键，因此 不符合BCNF。

✓ 分解为两张表:

课程安排表:

课程	教室
数据结构	101
数据库	101
操作系统	102

时间表:

课程	时间
数据结构	周一9:00
数据库	周三9:00
操作系统	周一9:00

这样，每个表中函数依赖左侧都是候选键，满足 BCNF。

六、练习题

题目：考虑下表是否满足BCNF？若不满足，请进行规范化。

员工编号	部门	经理
E01	财务	王总
E02	财务	王总
E03	市场	李总

已知依赖：

- 员工编号 $\rightarrow$ 部门 （主键）
- 部门 $\rightarrow$ 经理 （违反BCNF）

✓ 答案与解析:

- 员工编号 是主键, 部门 $\rightarrow$ 经理违反BCNF (因为部门不是候选键)
- 所以不满足BCNF

分解为:

1. 员工表 (员工编号, 部门)
2. 部门经理表 (部门, 经理)

✿ 七、总结

范式	主要问题解决	要求
1NF	原子性	每个字段值为原子值
2NF	部分依赖	每个非主属性完全依赖主键
3NF	传递依赖	非主属性只依赖主键
BCNF	主属性依赖其他主属性	所有依赖的左边必须是候选键

练习题（范式总结）

第一范式 (1NF)

练习题:

下表是否符合第一范式 (1NF)? 若不符合, 请说明原因并进行转换。

学号	姓名	电话号码
S001	张三	123456, 789101
S002	李四	222333

✅ 参考答案:

不符合第一范式。

🔍 解析:

问题：电话号码列包含多个值（123456 和 789101），违反了第一范式对原子性的要求。

规范化后的表：

学号	姓名	电话号码
S001	张三	123456
S001	张三	789101
S002	李四	222333

通过拆分多值字段为多行，使每个单元格保持“单值性”，即符合 1NF。

第二范式 (2NF)

练习题:

考虑如下关系 CourseSelection(学号, 课程号, 姓名, 课程名)
主键为: (学号, 课程号)

问: 此关系是否满足第二范式? 若不满足, 请规范化。

✓ 参考答案:

不满足第二范式。

🔍 解析:

分析依赖:

- 主键是 (学号, 课程号)
- 姓名 只依赖 学号 (部分依赖 ✗)
- 课程名 只依赖 课程号 (部分依赖 ✗)

规范化为两个表:

1. 学生表 (学号, 姓名)
2. 课程表 (课程号, 课程名)
3. 选课表 (学号, 课程号)

通过分解, 消除了非主属性对主键的部分依赖, 满足 2NF。

第三范式 (3NF)

练习题:

考虑关系 Employee(员工ID, 部门ID, 部门名称)

其中:

- 员工ID 是主键
- 员工ID $\rightarrow$ 部门ID
- 部门ID $\rightarrow$ 部门名称

该关系是否满足第三范式? 若不满足, 请规范化。

✅ 参考答案:

不满足第三范式。

🔍 解析:

分析:

- 员工ID 是主键
- 员工ID $\rightarrow$ 部门ID $\rightarrow$ 部门名称
 $\Rightarrow$ 部门名称 通过 部门ID 传递依赖于 员工ID, 违反 3NF

规范化为两个表:

1. 员工表 (员工ID, 部门ID)
2. 部门表 (部门ID, 部门名称)

消除传递依赖后, 满足 3NF。

Boyce-Codd范式 (BCNF)

练习题:

考虑如下关系 StudentCourse(学生, 课程, 教师)

已知函数依赖为:

- 学生 + 课程 $\rightarrow$ 教师
- 教师 $\rightarrow$ 课程

问: 此关系是否符合BCNF? 若不符合, 请规范化。

✓ 参考答案:

不符合BCNF。

🔍 解析:

分析:

- (学生, 课程) 是候选键 ✓
- 教师 $\rightarrow$ 课程, 但教师不是候选键 ✗

$\Rightarrow$ 不满足 BCNF 要求 (所有依赖左侧必须是候选键)

规范化为两个表:

1. 教师课程表 (教师, 课程)
2. 学生教师表 (学生, 教师)

这样所有函数依赖的左侧都是候选键, 符合 BCNF。

好的, 以下是一道关于关系模式分解的典型练习题, 附带参考答案和详细解析, 适合用于教学讲解或课堂练习。

模式分解练习题一

题目:

给定关系模式:

$R(\text{学号}, \text{姓名}, \text{院系}, \text{课程号}, \text{课程名}, \text{分数})$

已知函数依赖集:

- $F1: \text{学号} \rightarrow \text{姓名}, \text{院系}$
- $F2: \text{课程号} \rightarrow \text{课程名}$
- $F3: \text{学号}, \text{课程号} \rightarrow \text{分数}$

问题:

1. 判断该关系是否满足第三范式 (3NF) ?
2. 若不满足, 请将其分解为一组满足3NF的关系模式, 要求:
 - 保持无损连接性 (lossless join)
 - 保持依赖保持性 (dependency-preserving)

✅ 参考答案与解析:

🔍 第一步：确定主键

从依赖集中推理:

- 学号 + 课程号 $\rightarrow$ 分数
- 学号 $\rightarrow$ 姓名, 院系
- 课程号 $\rightarrow$ 课程名

因此:

候选键: (学号, 课程号) ✅

🔍 第二步：分析3NF

看每个非主属性是否完全依赖候选键，且没有传递依赖：

- 姓名、院系 $\leftarrow$ 学号 ✗ (传递依赖)
- 课程名 $\leftarrow$ 课程号 ✗ (传递依赖)

$\Rightarrow$ 所以该关系不满足3NF

🔧 第三步：分解为满足3NF的子模式

我们根据函数依赖将 R 分解如下：

子模式1: R1(学号, 姓名, 院系)

→ 对应依赖：学号 → 姓名, 院系 ✓

子模式2: R2(课程号, 课程名)

→ 对应依赖：课程号 → 课程名 ✓

子模式3: R3(学号, 课程号, 分数)

→ 对应依赖：学号, 课程号 → 分数 ✓ (原关系的主键)

🔍 验证:

✅ 无损连接性:

$$\begin{aligned} R1 \cap R3 &= \{\text{学号}\} \\ R2 \cap R3 &= \{\text{课程号}\} \\ R1 \cap R2 &= \emptyset \end{aligned}$$

并且:

- R1、R2、R3 的公共属性能通过候选键连接回原始关系 $\Rightarrow$ 无损连接性成立

✅ 依赖保持性:

原始依赖 $F = \{F1, F2, F3\}$ 全部在各子关系中可保留或推导 $\Rightarrow$ 依赖保持性成立

✓ 最终分解结果:

R1(学号, 姓名, 院系)
R2(课程号, 课程名)
R3(学号, 课程号, 分数)

每个子关系都满足第三范式, 同时保持无损连接和依赖保持, 是一次有效的3NF规范化分解。

高级模式分解练习题

无损连接性定义:

当一个关系模式 R 被分解成若干个子模式 $R_1, R_2, \dots, R_n$ 时, 如果我们可以通过对这些子模式进行自然连接 (natural join), 完全、准确地还原出原始关系 R 中的所有元组, 而不会产生额外的、虚假的元组, 就称这种分解具有“无损连接性”。

也称为:

- 无损分解 (lossless decomposition)
- 无损联接分解 (lossless-join decomposition)

✓ 数学形式定义:

给定关系模式 R 被分解为 $R_1, R_2, \dots, R_n$, 如果对于任意满足原有依赖集的关系实例 r :

$$\pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \cdots \bowtie \pi_{R_n}(r) = r$$

则称此分解为 无损连接分解 (Lossless Join Decomposition) 。

📌 为什么无损连接性很重要?

在数据库设计中，分解关系是为了消除冗余、异常，但：

- 如果分解损失了信息，就无法通过连接子关系还原原关系
- 这会导致数据丢失或查询错误

因此，无损连接性是规范化中不可妥协的基本要求

🧠 快速判断（两关系分解的情况）：

给定 R 被分为 R_1 和 R_2 ，分解是无损的充要条件为：

$$(R_1 \cap R_2) \rightarrow R_1 \quad \text{或} \quad (R_1 \cap R_2) \rightarrow R_2$$

即：共有属性能函数决定其中一个子关系的所有属性。

✓ 举例说明

原始关系 $R(A, B, C)$

函数依赖: $A \rightarrow B$

分解为:

- $R_1(A, B)$
- $R_2(A, C)$

交集是 $\{A\}$, 且 $A \rightarrow B$ 存在 $\Rightarrow$ 无损连接分解

依赖保持性定义

当一个关系模式 R 被分解为若干个子关系模式 $R_1, R_2, \dots, R_n$ 后，如果原关系 R 上的所有函数依赖都能够在这些子关系中直接或间接地被保持和推导出来，就称该分解具有“依赖保持性”（Dependency Preservation）。

✓ 形式化定义：

设原始关系模式为 R ，其函数依赖集为 F ，分解为子模式 $R_1, R_2, \dots, R_n$ 。记：

- F_i 是 F 中适用于 R_i 的依赖集合（即依赖左、右部都属于 R_i ）
- 令

$$F' = \bigcup F_i$$

如果：

$$F^+ = (F')^+$$

即：原依赖集的闭包 = 所有子关系中依赖的闭包

则分解为依赖保持的分解。

📌 为什么依赖保持性很重要?

- 在实际数据库操作中，我们希望在每个子关系上就能检查和维护数据一致性（依赖约束）。
- 如果依赖不能在局部关系中表达，就需要对多个表联合后进行检查，效率差、代价高。

✅ 依赖保持分解 $\Rightarrow$ 更容易实现约束检查

❌ 不保持依赖 $\Rightarrow$ 需频繁 join 验证，成本高

✅ 示例说明

原关系: $R(A, B, C)$

依赖集: $F = \{A \rightarrow B, B \rightarrow C\}$

分解为:

- $R_1(A, B)$
- $R_2(B, C)$

检查依赖保持性:

- R_1 保留 $A \rightarrow B$ ✅
- R_2 保留 $B \rightarrow C$ ✅
- $F' = \{A \rightarrow B, B \rightarrow C\}$
- $F'^+ = F^+ \Rightarrow$ ✅ 依赖保持

✗ 非依赖保持的例子

原关系: $R(A, B, C)$

$F = \{A \rightarrow B, B \rightarrow C\}$

分解为:

- $R_1(A, B)$
- $R_2(A, C)$

R_2 无法保存 $B \rightarrow C$, 也不能从 R_1 和 R_2 分别推导出 $B \rightarrow C$

$\Rightarrow$ ✗ 不保持依赖

✓ 小结对比

特性	依赖保持性	无损连接性
是否必须满足?	理想状态 (3NF 强调)	必须满足 (所有分解必须)
检查数据一致性	子关系中可直接检查	必须联合检查依赖, 效率低
在 BCNF 中能否总保持	✗ 不一定	✓ BCNF 强制保持无损连接

题目:

考虑关系模式: $R(A, B, C, D, E)$

函数依赖集 F :

- $A \rightarrow B$
- $B \rightarrow C$
- $A, D \rightarrow E$

问题:

1. 写出 R 的所有候选键
2. 判断该关系是否满足 BCNF?
3. 若不满足, 请将其分解为一组满足 BCNF 的子模式, 并说明是否:
 - 保持无损连接性
 - 保持依赖保持性

✓ 参考答案与解析

✓ 第一步：求候选键

已知依赖：

- $A \rightarrow B$
- $B \rightarrow C$
- $A, D \rightarrow E$

尝试闭包计算：

- $A^+ = \{A, B, C\}$
- $A, D^+ = \{A, D, B, C, E\} = \text{所有属性}$ ✓

$\Rightarrow$ 所以候选键是：(A, D)

✅ 第二步：判断是否满足 BCNF

我们检查每个函数依赖的左部是否是候选键：

函数依赖 左部是候选键？ 是否满足BCNF？

$A \rightarrow B$ 否 ❌ 违反BCNF

$B \rightarrow C$ 否 ❌ 违反BCNF

$A, D \rightarrow E$ 是 ✅ 满足BCNF

❌ 所以 R 不满足 BCNF，要进行分解。

✓ 第三步：BCNF 分解

第一步：以 $A \rightarrow B$ 分解

1. $R_1(A, B)$

- $A \rightarrow B$
- A 是主键 $\Rightarrow$ 满足 BCNF ✓

2. $R_2(A, C, D, E)$

- 在 R_2 中，保留依赖：
- $B \rightarrow C$ 不存在（ B 不在 R_2 ）
- $A, D \rightarrow E$ 保留

第二步：处理 $R_2(A, C, D, E)$

- 重新判断依赖： $A, D \rightarrow E$
- A, D 是 R_2 的候选键
- 依赖满足 BCNF 
- 但我们还要检查： $A \rightarrow B \rightarrow C$ 推出 C 是否有依赖
- 目前 C 不是任何右部的函数依赖

$\Rightarrow R_2$ 不再违反 BCNF

✅ 第四步：最终分解结果（BCNF）

$R_1(A, B)$

$R_2(A, C, D, E)$

✅ 第五步：验证属性保留与连接性

✅ 无损连接性：

- $R1 \cap R2 = \{A\}$
- A 是 R1 的主键 ✅

⇒ 满足无损连接条件（公共属性为一个关系的主键）

❌ 依赖保持性？

原依赖 $F = \{A \rightarrow B, B \rightarrow C, A, D \rightarrow E\}$

- $A \rightarrow B$ ✓ 保留在 R1
- $A, D \rightarrow E$ ✓ 保留在 R2
- ❌ $B \rightarrow C$ 无法在任一子关系中单独保留

⇒ 依赖保持性不满足 ❌（在 BCNF 中常常无法同时满足）

✅ 最终总结:

项目	结果
候选键	(A, D)
是否满足 BCNF	否 (因 $A \rightarrow B, B \rightarrow C$)
BCNF 分解后的关系	$R_1(A, B), R_2(A, C, D, E)$
无损连接性	✅ 是
依赖保持性	❌ 否